# LangChain & LangGraph Migration Guide

Migration knowledge document for testing an autonomous Python code migration agent.

## Purpose

This document contains explicit migration rules. A migration agent should extract these rules from the documentation, identify matching constructs in Python source code, retrieve supporting documentation, and apply transformations without inventing unsupported rules.

## Rule 1 — OpenAI LLM import

**Legacy:**

```
from langchain.llms import OpenAI
```

**Modern:**

```
from langchain_openai import OpenAI
```

Migration note: Apply this rule only when the source construct matches the documented legacy form.

## Rule 2 — ChatOpenAI import

**Legacy:**

```
from langchain.chat_models import ChatOpenAI
```

**Modern:**

```
from langchain_openai import ChatOpenAI
```

Migration note: Apply this rule only when the source construct matches the documented legacy form.

## Rule 3 — OpenAI embeddings

**Legacy:**

```
from langchain.embeddings import OpenAIEmbeddings
```

**Modern:**

```
from langchain_openai import OpenAIEmbeddings
```

Migration note: Apply this rule only when the source construct matches the documented legacy form.

## Rule 4 — Chroma integration

**Legacy:**

```
from langchain.vectorstores import Chroma
```

**Modern:**

```
from langchain_chroma import Chroma
```

Migration note: Apply this rule only when the source construct matches the documented legacy form.

## Rule 5 — FAISS integration

**Legacy:**

```
from langchain.vectorstores import FAISS
```

**Modern:**

```
from langchain_community.vectorstores import FAISS
```

Migration note: Apply this rule only when the source construct matches the documented legacy form.

## Rule 6 — Hugging Face embeddings

**Legacy:**

```
from langchain.embeddings import HuggingFaceEmbeddings
```

**Modern:**

```
from langchain_community.embeddings import HuggingFaceEmbeddings
```

Migration note: Apply this rule only when the source construct matches the documented legacy form.

## Rule 7 — Document loaders

**Legacy:**

```
from langchain.document_loaders import PyPDFLoader
```

**Modern:**

```
from langchain_community.document_loaders import PyPDFLoader
```

Migration note: Apply this rule only when the source construct matches the documented legacy form.

## Rule 8 — Text splitters

**Legacy:**

```
from langchain.text_splitter import RecursiveCharacterTextSplitter
```

**Modern:**

```
from langchain_text_splitters import RecursiveCharacterTextSplitter
```

Migration note: Apply this rule only when the source construct matches the documented legacy form.

## Rule 9 — Prompt templates

**Legacy:**

```
from langchain.prompts import PromptTemplate
```

**Modern:**

```
from langchain_core.prompts import PromptTemplate
```

Migration note: Apply this rule only when the source construct matches the documented legacy form.

## Rule 10 — Output parsers

**Legacy:**

```
from langchain.output_parsers import StrOutputParser
```

**Modern:**

```
from langchain_core.output_parsers import StrOutputParser
```

Migration note: Apply this rule only when the source construct matches the documented legacy form.

## Rule 11 — LLM chains

**Legacy:**

```
from langchain.chains import LLMChain
```

**Modern:**

```
Use an LCEL RunnableSequence pipeline instead of the legacy LLMChain abstraction.
```

Migration note: Apply this rule only when the source construct matches the documented legacy form.

## Rule 12 — Agent initialization

**Legacy:**

```
from langchain.agents import initialize_agent
```

**Modern:**

```
Use create_agent from langchain.agents for the current agent API.
```

Migration note: Apply this rule only when the source construct matches the documented legacy form.

## Rule 13 — LangGraph ReAct agent

**Legacy:**

```
from langgraph.prebuilt import create_react_agent
```

**Modern:**

```
Use the current LangChain/LangGraph agent API; do not introduce new code using the
deprecated prebuilt helper.
```

Migration note: Apply this rule only when the source construct matches the documented legacy form.

## Rule 14 — LangGraph StateGraph import

**Legacy:**

```
from langgraph.graph import StateGraph
```

**Modern:**

```
from langgraph.graph import StateGraph
```

Migration note: Apply this rule only when the source construct matches the documented legacy form.

## Rule 15 — LangGraph START/END

**Legacy:**

```
from langgraph.graph import START, END
```

**Modern:**

```
from langgraph.graph import START, END
```

Migration note: Apply this rule only when the source construct matches the documented legacy form.

## Example Combined Migration

**Legacy code:**

```
from langchain.chat_models import ChatOpenAI
from langchain.embeddings import OpenAIEmbeddings
from langchain.vectorstores import FAISS
from langchain.text_splitter import RecursiveCharacterTextSplitter

llm = ChatOpenAI()
embeddings = OpenAIEmbeddings()
vectorstore = FAISS.from_documents([], embeddings)
splitter = RecursiveCharacterTextSplitter(chunk_size=500)
```

**Expected migrated imports:**

```
from langchain_openai import ChatOpenAI, OpenAIEmbeddings
from langchain_community.vectorstores import FAISS
from langchain_text_splitters import RecursiveCharacterTextSplitter
```

## Agent Behavior Requirements

• Extract only migration rules explicitly supported by this document.

• Do not infer or invent a replacement when the document does not provide one.

• A CST node becomes a migration candidate when it matches an extracted rule.

• Use semantic retrieval to obtain supporting documentation for a candidate.

• Use the LLM to reason about the exact transformation, then apply the approved transformation with LibCST.

• Preserve program behavior, comments, and unrelated identifiers whenever possible.